

Pràctica 1

1. Generació del codi

El codi de la primera pràctica es el següent:

```
#include <Arduino.h>
#define LED_PIN 2 // Definim el pin LED integrat

// Configuració inicial (se executa UNA sola vegada)
void setup() {
  Serial.begin(115200); // Iniciar comunicacion serie a 115200 baudios
  pinMode(LED_PIN, OUTPUT); // Configurem el pin LED com SORTIDA

  // Mensaje inicial por el Monitor Serie
  Serial.println("Programa iniciado!");
  Serial.println("El LED va a parpadear...");
}

// Bucle principal (es repeteix inifinitament)
void loop() {
  digitalWrite(LED_PIN, HIGH); // Encender el LED
  Serial.println("LED ENCENDIDO");

  // Esperar 1 segundo (1000 milisegundos)
  delay(1000);
  digitalWrite(LED_PIN, LOW); // Apagar el LED
  Serial.println("LED APAGADO");

  // Esperar 1 segundo
  delay(1000);
}
```

2. Diagrama de flux i diagrama de temps

Diagrama de flux

Seguidament, hem realitzat el diagrama de flux de la primera pràctica, aquest consta de dos blocs principals, el bloc de configuració i el bucle principal. Aquests dos blocs constan a més, del seus propis sub blocs.

Diagrama de flux: P1

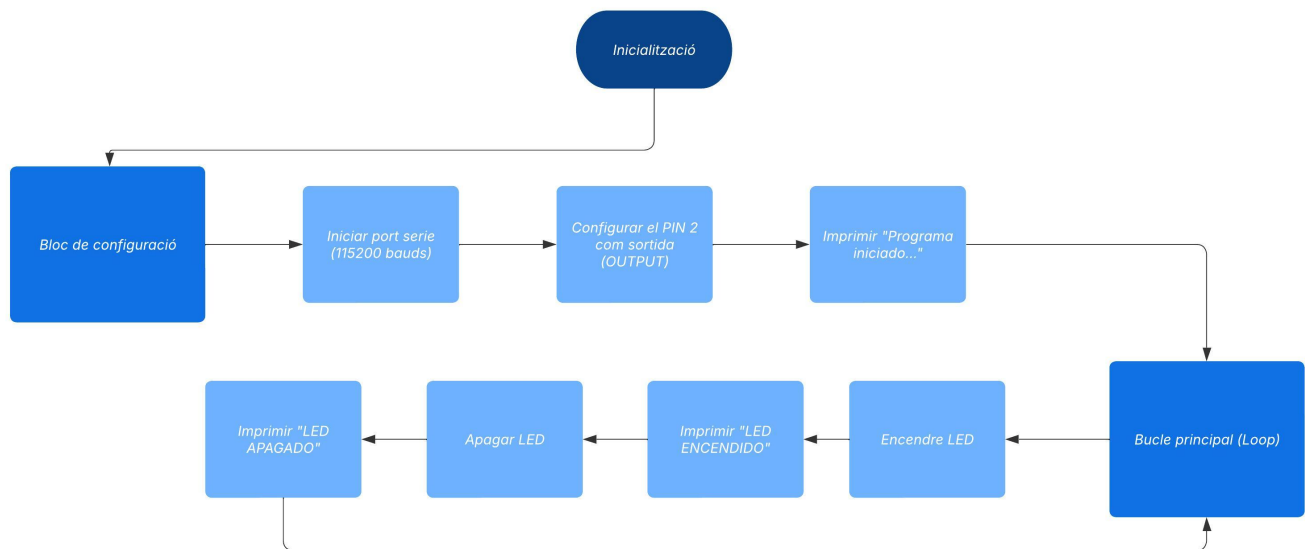
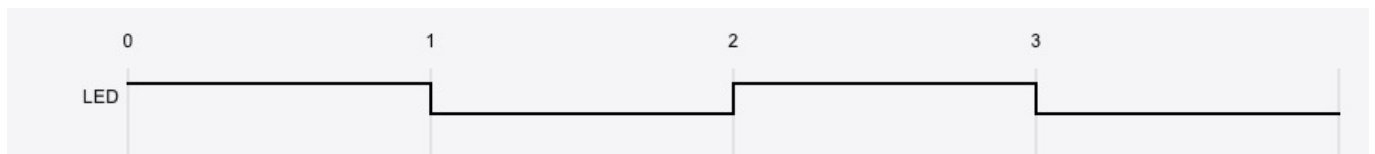


Diagrama de temps

Per finalitzar aquest apartat, hem disñat el diagrama de temps de la primera pràctica, on la continuïtat del nostre LED es senzilla. El programa comença encenen el LED, espera un 1000 ms, seguidament s'apaga el LED y es manté 1000 ms i així repetidament.



3. Pregunta

En el prgrama que se ha realitzat, quin es el temps lliure que té el processador?

En el cas d'aquest programa, el temps lliure que el nostre processador és pràcticament nul. Això és a causa de la comanda `delay()`. El microcontrolador es manté ocupat al 100% executant el bucle intern d'espera, la qual cosa impedeix que el processador quedi lliure per dur a terme qualsevol altra tasca o procés paral·lel fins que finalitzi el temps indicat.

Practica 1: Pujada de nota

Codi de la pujada de nota

El codi de la pujada de nota de la Práctica 1 es el següent:

```
#include <Arduino.h>
const int potPin = 7; // El potenciómetro esta conectado al GPIO 34
                        (Analog ADC1_CH6)

int potValue = 0; // La variable para guardar el valor del
                  potenciómetro

void setup() {
  Serial.begin(115200);
  delay(1000);
}

void loop() {
  // Reading potentiometer value
  potValue = analogRead(potPin);
  Serial.println(potValue);
  delay(500);
}
```

Diagrama de flux

Diagrama de flux: P1_pujarnota

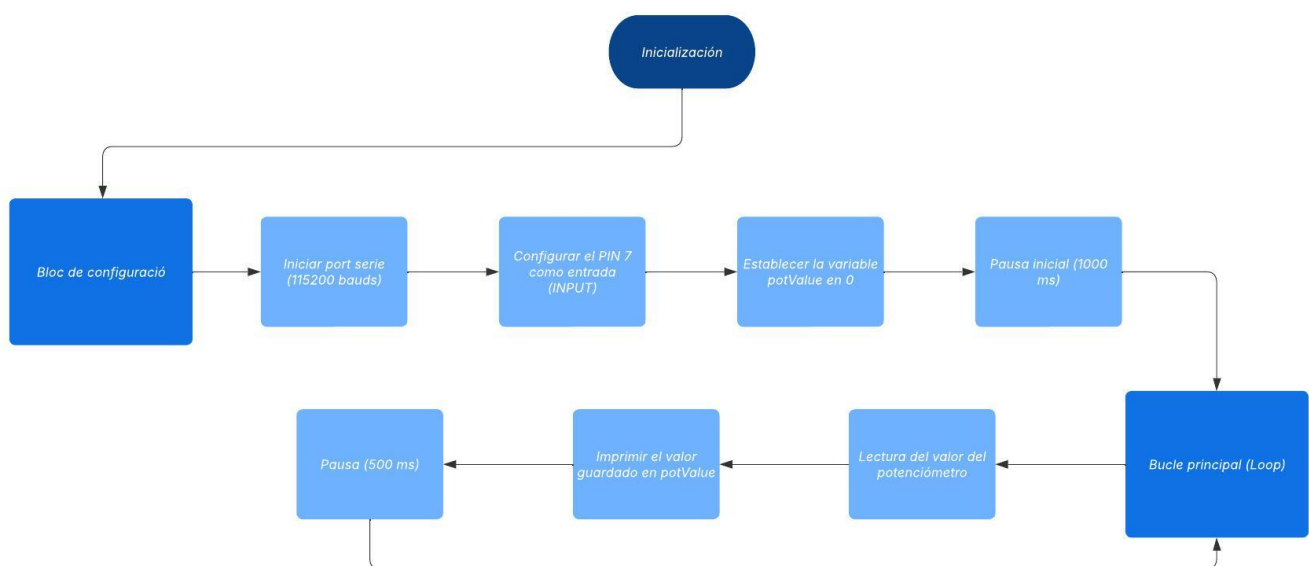


Diagrama de temps



Practica 1: Part complementaria

Codi de la Pràctica complementaria

El codi de la la part complementaria de la Práctica 1 es el següent:

```
#include <Arduino.h>
#include <Adafruit_NeoPixel.h> //Incluimos la librería
Adafruit_NeoPixel

// pin del led RGB en PCB
#define PIN 48 // ESP32S3 Generic -n8r2-

// Un solo led en PCB
#define NUMPIXELS 1

// Inicializamos el objeto "pixels"
// Argumento 1 = número de pixeles
// Argumento 2 = número del pin de datos
// Argumento 3 = tipo de pixel: NEO_KHZ800 800 KHz bitstream (most
NeoPixel products w/> WS2812 LEDs)
Adafruit_NeoPixel pixels(NUMPIXELS, PIN, NEO_GRB + NEO_KHZ800);
// Time (en milliseconds) pausa entre pixels
#define DELAYVAL 1000

void setup() {
    delay(500);
    // Inicializamos el objeto pixels
    pixels.begin();
    Serial.begin(115200);
    delay(500);
    // Información chip
    Serial.println("\n\n=====");
    Serial.printf("Chip Modelo: %s\n", ESP.getChipModel());
    Serial.printf("Chip Revision: %d\n", ESP.getChipRevision());
    Serial.printf("E %d core\n", ESP.getChipCores());
    Serial.printf("Flash Chip capacidad: %d \n",
ESP.getFlashChipSize());
    Serial.printf("Flash Chip velocidad: %d \n",
ESP.getFlashChipSpeed());

    esp_chip_info_t chip_info;
    esp_chip_info(&chip_info);
    Serial.printf("\nCaracterísticas:\n %s\n %s\n %s\n %s\n %s\n",
        (chip_info.features & CHIP_FEATURE_EMB_FLASH) ? "flash en
ESP32" : "Sin flash en > ESP32",
        (chip_info.features & CHIP_FEATURE_WIFI_BGN) ? "2.4GHz WiFi" :
"Sin Wifi",
        (chip_info.features & CHIP_FEATURE_BLE) ? "Bluetooth LE" : "Sin
Bluetooth LE",
```

```
        (chip_info.features & CHIP_FEATURE_BT) ? "Bluetooth" : "Sin
Bluetooth",
        (chip_info.features & CHIP_FEATURE_IEEE802154) ? "IEEE
802.15.4" : "NO IEEE 802.> 15.4");
    // MAC Address
    String MACString = "";
    uint64_t chipid = ESP.getEfuseMac();
    for (int i=0; i<6; i++) {
        if (i > 0) MACString.concat(":");
        uint8_t Octet = chipid >> (i * 8);
        MACString.concat(String(Octet, HEX));
    }
    Serial.println("MAC: " + MACString);
}

void loop() {
    // apagamos
    pixels.clear();
    pixels.show();
    Serial.println("");
    delay(1000);

    // Encendemos con el color rojo, con brillo al máximo
    pixels.setPixelColor(0, pixels.Color(255, 0, 0));
    // enviamos
    pixels.show();
    Serial.println("R");
    delay(DELAYVAL);

    // Encendemos con el color verde, con brillo moderado
    pixels.setPixelColor(0, pixels.Color(0, 130, 0));
    pixels.show();
    Serial.println("G");
    delay(DELAYVAL);

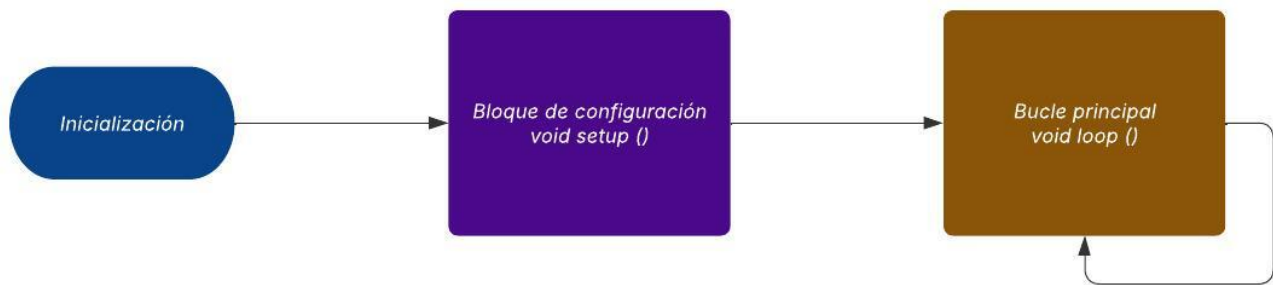
    // Encendemos con el color azul, con brillo moderado
    pixels.setPixelColor(0, pixels.Color(0, 0, 130));
    pixels.show();
    Serial.println("B");
    delay(DELAYVAL);
}
```

2. Diagrama de flux i diagrama de temps

Diagrama de flux

Pel diagrama de flux, a causa del augment de complexitat del codi, hem realitzat dos 3 diagrames de flux, en aquest primer podem veure el flux general del programa

Diagrama de flux: P1_complementaria



Seguidament, em realitzat dos diagrames de flux diferents, un on podem veure el flux que segueix el primer bloc (el bloc de configuració) i el segon bloc (el bloc de execució o bucle principal).

En el nostre primer bloc, podem veure com la nostra placa ESP32 S3 està configurant-se, utilitzant les funcions d'una libreria externa per obtenir els següents elements:

- Modelo y revisión del chip (ESP32-S3)
- Número de núcleos y tamaño de la memoria Flash
- Características de conectividad (WiFi, Bluetooth, etc.)

En el nostre segon bloc es realitza el bucle o cicle principal, on essencialment es podrà veure com s'encén i espera el LED

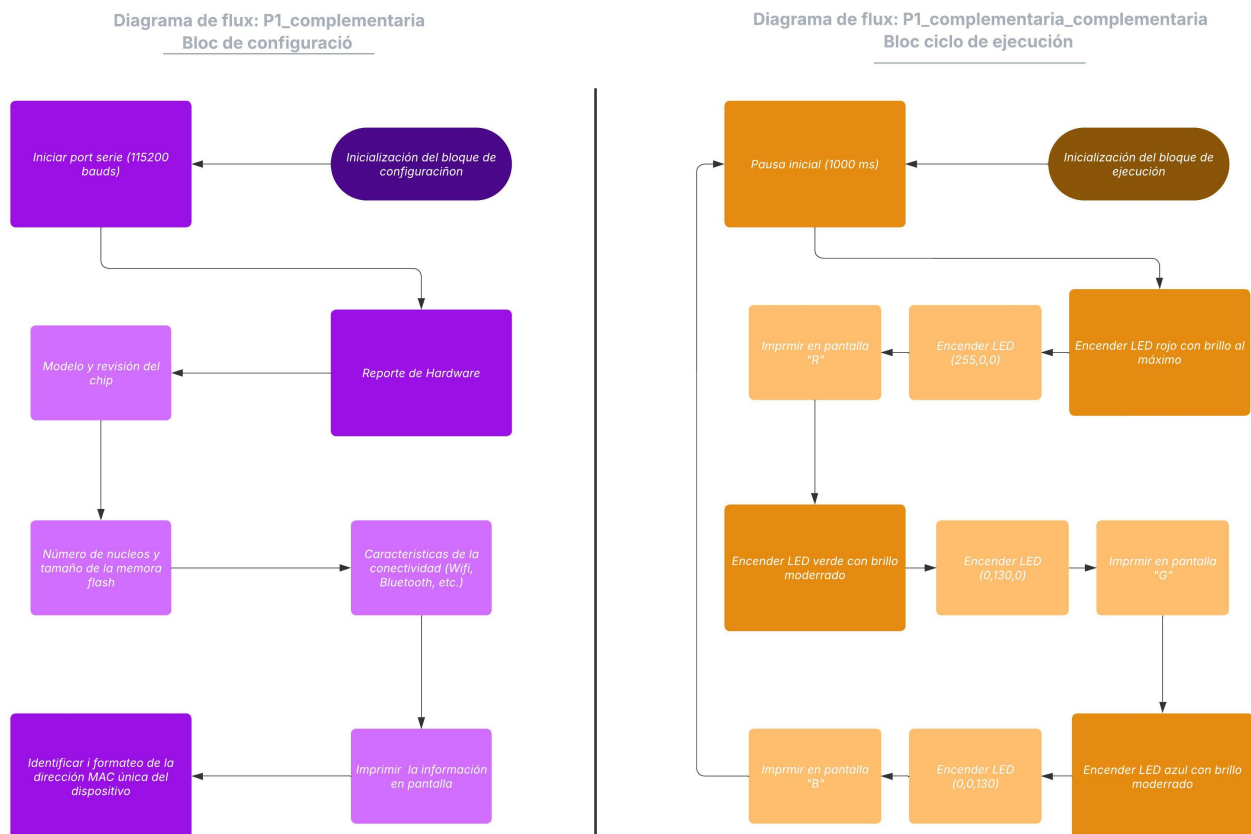


Diagrama de temps

Per el cas del nostre diagrama de temps, podem veure que el nostre LED comença amb una pausa inicial de 1000 ms i seguidament amb una serie completa ences, on aixó es pot veure reflectit en la part del programa següent:

```

void loop() {
  // apagamos
  pixels.clear();
  pixels.show();
  Serial.println("");
  delay(1000);

  // Encendemos con el color rojo, con brillo al máximo
  pixels.setPixelColor(0, pixels.Color(255, 0, 0));
  // enviamos
  pixels.show();
  Serial.println("R");
  delay(DELAYVAL);

  // Encendemos con el color verde, con brillo moderado
  pixels.setPixelColor(0, pixels.Color(0, 130, 0));
  pixels.show();
  Serial.println("G");
  delay(DELAYVAL);
}
  
```



```
// Encendemos con el color azul, con brillo moderado
pixels.setPixelColor(0, pixels.Color(0, 0, 130));
pixels.show();
Serial.println("B");
delay(DELAYVAL);

}
```

Com podem notar, després del milisegon 1000, el nostre LED no es torna a apagar fins que tornem al principi del `loop()`.



3. Modificació del programa

Modificar el programa para que visualice los 6 colores básicos (RGBCMY) durante un segundo y finalmente el blanco. Todos ellos con una intensidad lumínica mínima.

Per aquesta part, modifiquem el bucle principal, on es dur a terme la serie de colors del LED, afegim els 3 colors secundaris i el blanc.

```
void loop() {
  // apagamos
  pixels.clear();
  pixels.show();
  Serial.println("");
  delay(1000);

  // encendemos con el color Rojo
  pixels.setPixelColor(0, pixels.Color(1, 0, 0));
  // enviamos
  pixels.show();
  Serial.println("R");
  delay(DELAYVAL);

  // encendemos con el color Verde, con brillo minimo
  pixels.setPixelColor(0, pixels.Color(0, 1, 0));
  pixels.show();
  Serial.println("G");
  delay(DELAYVAL);

  // encendemos con el color Azul, con brillo minimo
  pixels.setPixelColor(0, pixels.Color(0, 0, 1));
  pixels.show();
}
```

```
Serial.println("B");
delay(DELAYVAL);

// encendemos con el color Cian, con brillo minimo
pixels.setPixelColor(0, pixels.Color(0, 1, 1));
pixels.show();
Serial.println("C");
delay(DELAYVAL);

// encendemos con el color Magenta, con brillo minimo
pixels.setPixelColor(0, pixels.Color(1, 0, 1));
pixels.show();
Serial.println("M");
delay(DELAYVAL);

// encendemos con el color Amarillo, con brillo minimo
pixels.setPixelColor(0, pixels.Color(1, 1, 0));
pixels.show();
Serial.println("Y");
delay(DELAYVAL);

// encendemos con el color Blanco, con brillo minimo
pixels.setPixelColor(0, pixels.Color(1, 1, 1));
pixels.show();
Serial.println("W");
delay(DELAYVAL);
}
```